



Ottimizzazione 4 - La gerarchia della Cache

Principi della Programmazione Parallela
CdL Informatica - A.A. 2025/2026



Università
di Catania

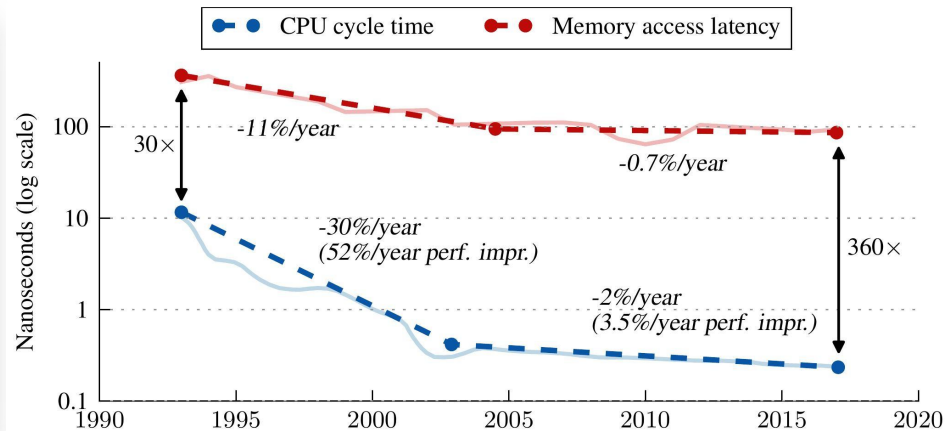
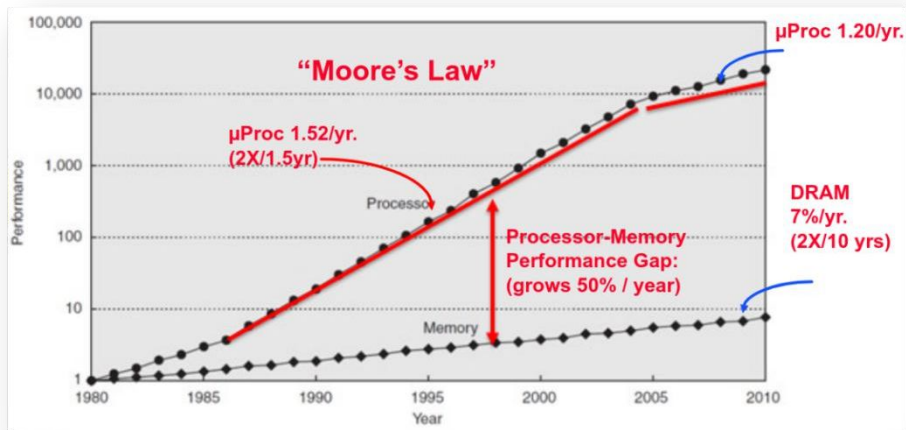


INAF
ISTITUTO NAZIONALE
DI ASTROFISICA

**Iniziamo con un piccolo riepilogo sulla
memoria e sulle cache**



Inizi anni 90: la CPU diventa più veloce della memoria



Al giorno d'oggi non c'è più alcun problema nella frequenza di clock. Tuttavia, la latenza del DRAM è ancora un problema serio.

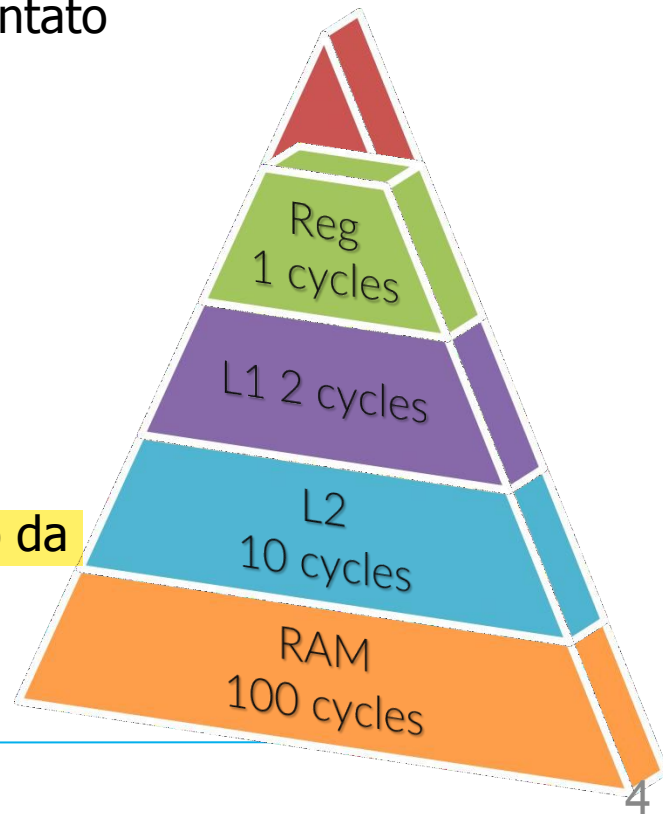


La memoria cache

Abbiamo visto che le CPU moderne hanno implementato una memoria SRAM speciale denominata **cache**.

La cache stessa ha una gerarchia:

- *Il livello I* si trova all'interno di ciascun core.
- *il livello II* si trova all'interno del core oppure può essere condiviso da più core.
- *Il livello III* è all'interno della CPU ed è condiviso da molti core.





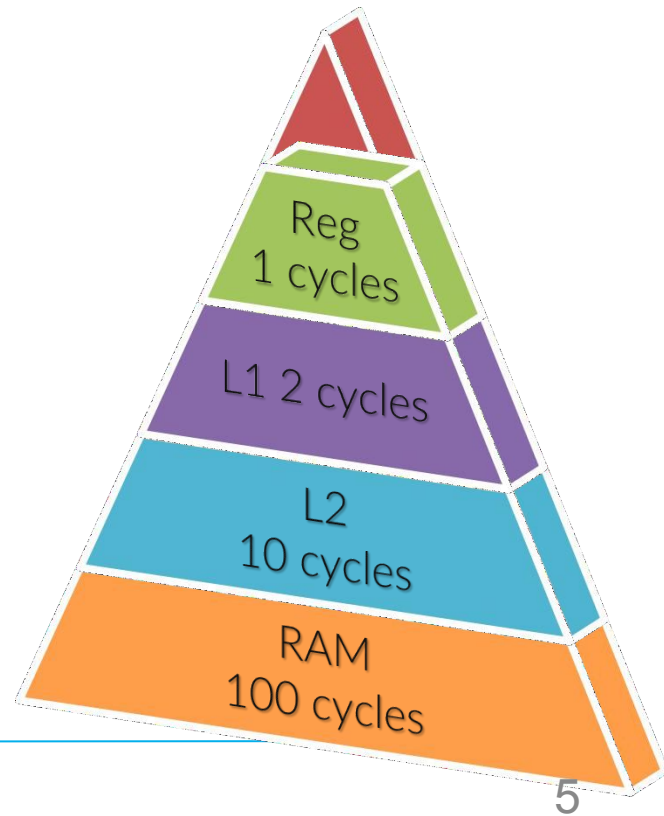
La memoria cache

Supponiamo di avere solo L1 e RAM.
Quale sarebbe il costo medio, *in cicli di CPU*, per recuperare i dati in funzione del livello di occupazione della cache L1?

L1 cache + RAM

$$L_{1\text{ costo}} + \text{Miss}_1 \times \text{RAM}_{\text{costo}}$$

- 100% di successo L1 -> 2 cicli
 - 99% di successo L1 -> 3 cicli
 - 97% di successo L1 -> 5 cicli
- } PIÙ LENTO DEL
50% - 150%





La memoria cache

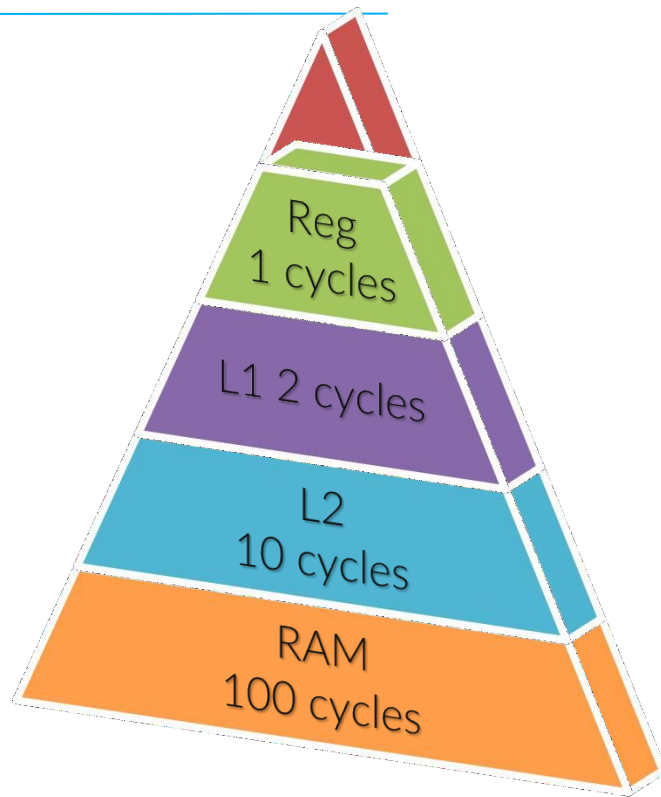
Aggiungiamo ora un 2 ° livello di cache, L2:

L1 cache + L2 cache + RAM

$$L_{1\text{ costo}} + \text{Miss}_1 \times (L2_{\text{costo}} + \text{Miss}_2 \times RAM_{\text{costo}})$$

- 100% di successo L1 -> 2 cicli
- 99% di successo L1, 100% di successo L2 -> 2.1 cicli
- 97% di successo L1, 100% di successo L2 -> 2.3 cicli
- 90% di successo L1, 97% di successo L2 -> 3.3 cicli

La media dei cicli per carico è molto migliore ora con un L2 più grande (oggi si trova anche un L3 ancora più grande)





Il principio di località

Siamo portati naturalmente al "principio di località":

I dati sono definiti "locali" quando risiedono in una porzione "piccola" dello spazio degli indirizzi a cui si accede in un periodo di tempo "breve"

-> è probabile che i dati locali siano nella cache

**Località
temporale**

se un indirizzo viene referenziato, è probabile che venga referenziato di nuovo presto

**Località
spaziale**

se un indirizzo è referenziato, è probabile che gli indirizzi vicini saranno referenziati presto



Mappatura Memoria → Cache

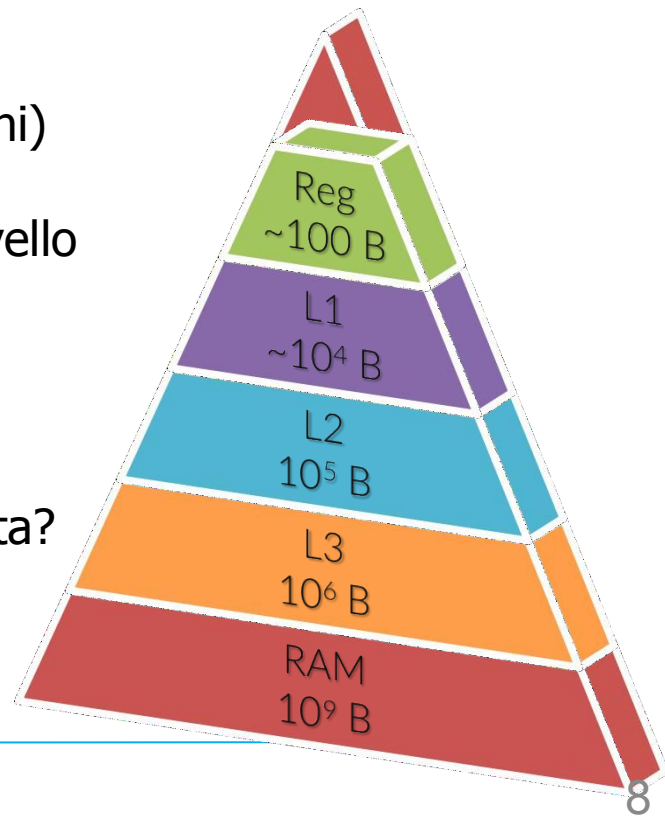
Domanda:

La RAM contiene $\sim 10^9$ byte, mentre L1 contiene $\sim 10^4$ byte (32 KB per i dati e 32 KB per le istruzioni)

Quindi, come si fa a mappare la RAM in un dato livello di cache, ad esempio L1, in modo efficace?

I problemi principali sono:

- Dove mappare un indirizzo
- Cosa succede se la posizione in L1 è già occupata?





Cache e
memoria

Mappatura Memoria → Cache

Supponiamo che sia la RAM che la cache siano suddivise in blocchi di uguale dimensione (ad esempio, 64B): non carichi solo un byte nella tua cache ma un intero blocco (normalmente chiamato *linea*)

Memoria
RAM

Numero di
blocco

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F

cache

numero di
blocco della
cache

numero del blocco
di memoria

dati

bit valid

bit dirty

0	1	2	3	4	5	6	7
.	.	.	.	.	.	.	.
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Ogni "blocco" qui è lungo 64B



Cache e
memoria

Mappatura Memoria → Cache

Mappatura completamente associativa

I dati possono essere inseriti in qualsiasi blocco di cache libero

cache

0	1	2	3	4	5	6	7

Mappatura diretta

I dati possono essere inseriti solo in un dato blocco cioè $\text{block_num} \% \text{blocks_in_cache}$

0	1	2	3	4	5	6	7

associativo n-way

I dati possono essere inseriti in pochi blocchi di cache, ad esempio

$\text{numero_blocco} \% (\text{blocchi_nella_cache} / n)$
 $n=2$

0	1	2	3	4	5	6	7

Le mappature dirette o associative n-way danno un aiuto nel capire se un blocco c'è o no in cache senza doverli analizzare tutti

Memoria
RAM

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F



Cache e
memoria

Mappatura memoria → cache

Mappatura completamente associativa

I dati possono essere inseriti in qualsiasi blocco di cache libero.

Pro

Molto efficiente nella scrittura (riduce al minimo i conflitti).

Usi al massimo la memoria

Contro

Molto inefficiente nella lettura (tutte le posizioni potrebbero contenere i dati indirizzati).

Mappatura diretta

I dati possono essere inseriti solo in un dato blocco



Pro

Molto efficiente nella scrittura (nessuna ricerca di posizioni disponibili).

Contro

Massimizza i conflitti della cache, poiché solo 1 posizione è idonea per un gran numero di indirizzi.

associativo n-way

I dati possono essere inseriti in pochi blocchi di cache



Pro

Efficiente nel determinare il posizionamento. Minore quantità di conflitti nella cache.

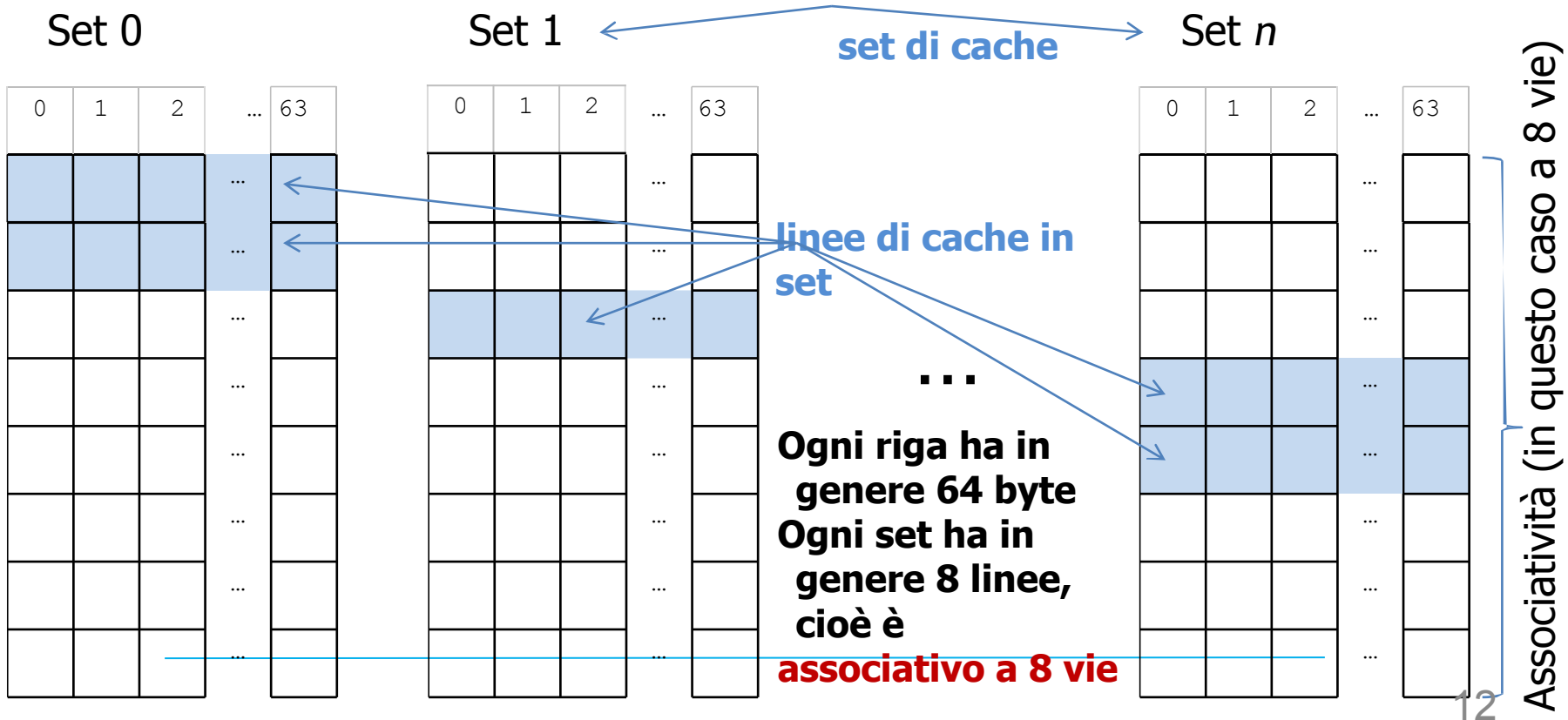
Contro

Logiche più complesse; maggior numero di operazioni per l'accesso.



Cache e
memoria

Una tipica cache odierna





Riepilogo della cache in due slide

**3C per i
nemici**

► **Compulsory miss**

Errori inevitabili quando i dati vengono letti per la prima volta

► **Capacity miss**

- Spazio insufficiente per contenere tutti i dati
- Troppi dati a cui si accede tra un utilizzo e l'altro

► **Conflict miss**

Ripulitura della cache a causa della mappatura dei dati sulle stesse linee della cache



Riepilogo della cache in due slide

3R per gli amici

Evitiamo continui cambi
di contesto nella cache
lavorando sempre con gli
stessi dati

► Riorganizza (codice e dati)

Progettare il layout per migliorare la localizzazione
temporale e spaziale

► Ridurre (dimensione)

- Dimensioni dei dati più piccole: blocchi più piccoli a cui si accede Sfrutti al meglio il blocco della cache
- Meno istruzioni

► Riutilizzo (linee di cache)

Aumentare la località spaziale e temporale:
conservare i dati residenti per svolgere più operazioni



Come ottenere le dimensioni della cache

Linux

```
$ lscpu
```

macOS

```
$ sysctl -a | grep cache
```

Cache	Ordine di grandezza
L1	~32 KB
L2	~256 KB
L3	~MB



Il modello di accesso alla memoria

Quanto velocemente posso leggere i dati dalla memoria?

Uno strumento per **capire e visualizzare** le prestazioni della memoria è il **Memory Mountain**, una visualizzazione delle prestazioni della memoria in funzione di:

- Dimensione del working set (quanti dati uso)
- Stride (come accedo ai dati)
- Bandwidth (MB/s)

Dati piccoli + accesso sequenziale → veloce

Dati grandi + accesso sparso → lento



Il modello di accesso alla memoria

```
#define N 1000000
```

```
int a[N];  
int sum = 0;
```

```
/* Accesso sequenziale (stride = 1) */  
for (int i = 0; i < N; i++) {  
    sum += a[i];  
}
```

```
/* Accesso con stride grande */  
for (int i = 0; i < N; i += 16) {  
    sum += a[i];  
}
```

Quale è più veloce?



Il modello di accesso alla memoria

```
#define N 1000000
int a[N];
int sum = 0;

/* Accesso sequenziale (stride = 1) */
for (int i = 0; i < N; i++) {
    sum += a[i];
}

/* Accesso con stride grande */
for (int i = 0; i < N; i += 16) {
    sum += a[i];
}
```

Naturalmente, l'accesso sequenziale è meglio in lettura perché i dati sono sequenziali. Andando a salti, abbiamo degli sprechi.

- La CPU carica **linee di cache** (es. 64 byte)
- Accesso sequenziale:
 - Usa tutti i dati caricati → efficiente
- Accesso con stride:
 - Spreca quasi tutta la cache line



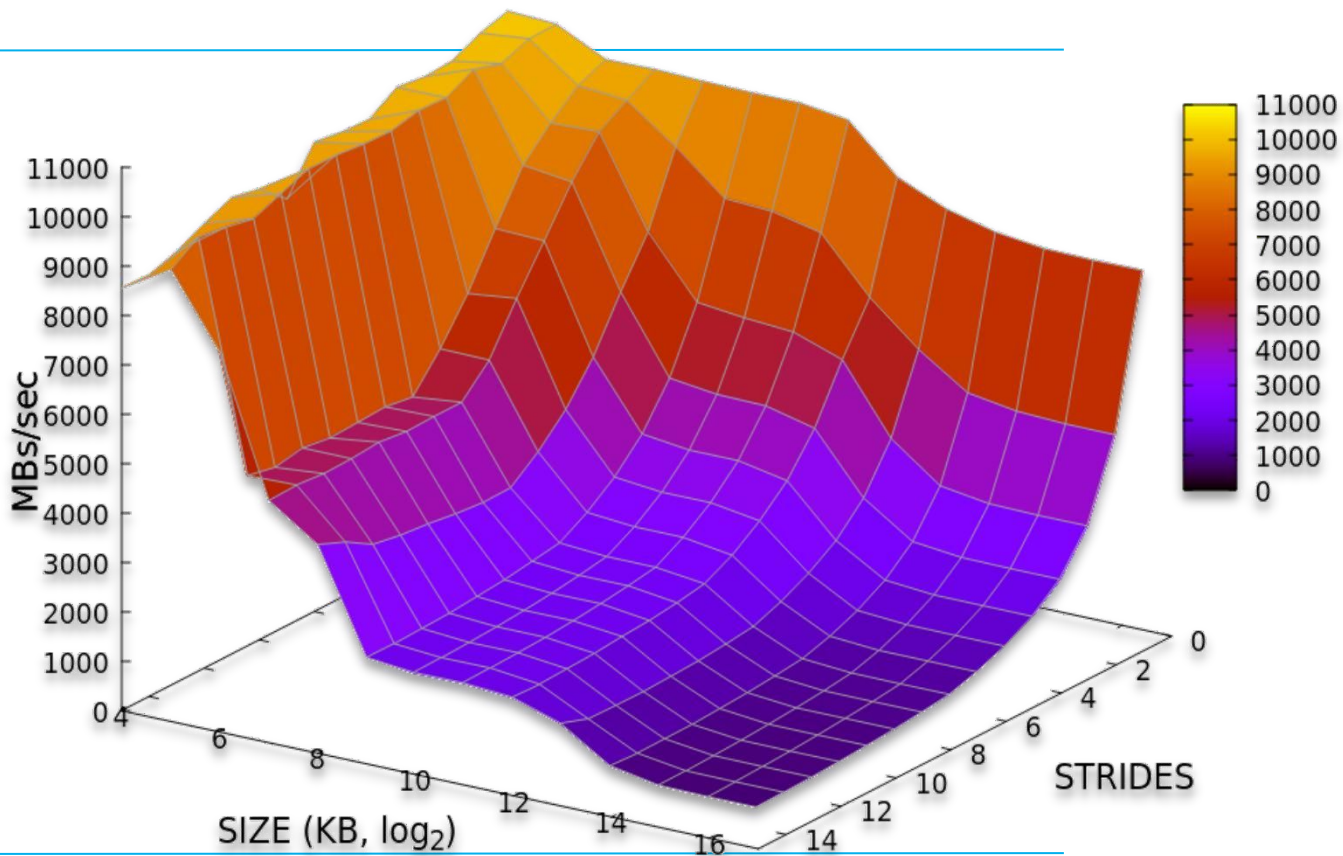
Stesso codice, prestazioni molto diverse



Cache e
memoria

Il modello di accesso alla memoria

**Il risultato è...
la montagna della
memoria**

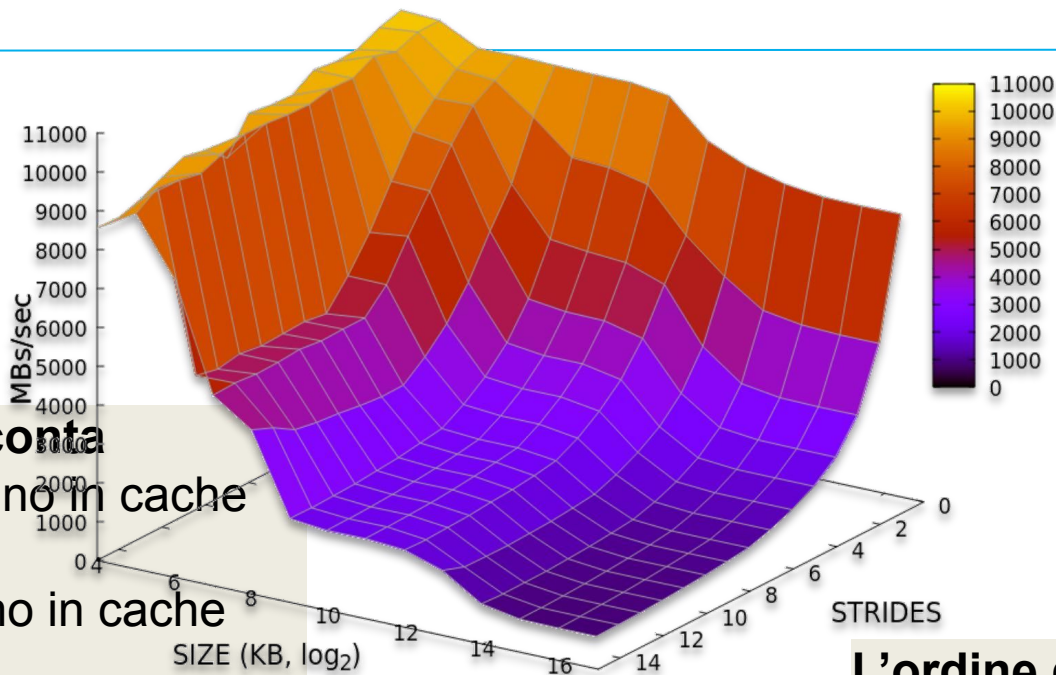


memory_mountain/mountain.c



Cache e
memoria

Il modello di accesso alla memoria



La dimensione conta

- Se i dati stanno in cache
→ **veloci**
- Se non stanno in cache
→ **lenti**

Regole d'oro

- ✓ Usa accessi sequenziali
- ✓ Mantieni piccoli i working set
- ✓ Riusa i dati mentre sono in cache

L'ordine di accesso conta

- Stride piccolo
→ alta bandwidth
- Stride grande
→ crollo delle prestazioni

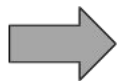


Accesso a grandi passi

Consideriamo un problema abbastanza comune: la trasposta di una matrice.

**Trasposta della
matrice**

A_{00}	A_{01}	...	A_{0N}
A_{10}	A_{11}	...	A_{1N}
...	...	...	...
A_{N0}	A_{N1}	...	A_{NN}



B_{00}	B_{01}	...	B_{0N}
B_{10}	B_{11}	...	B_{1N}
...	...	...	...
B_{N0}	B_{N1}	...	B_{NN}

=

A_{00}	A_{10}	...	A_{N0}
A_{01}	A_{11}	...	A_{N1}
...	...	...	...
A_{0N}	A_{1N}	...	A_{NN}

$$A_{ij}^T = A_{ji}$$



Accesso a grandi passi

L'implementazione molto semplice e diretta è:

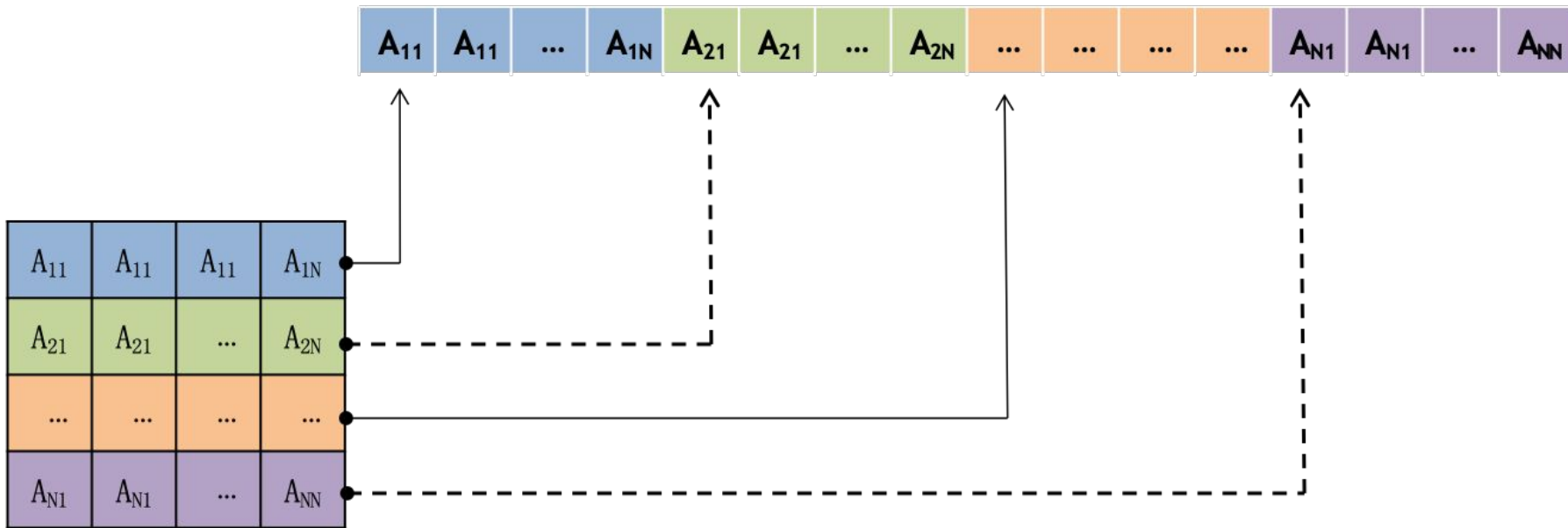
Trasposta della matrice

```
for(int row = 0; row < N;  
row++)  
    for(col = 0; col < N;  
col++)  
        A [ col*N + row ] = B [ row*N + col ];
```



Nota: come una matrice viene memorizzata nella memoria

Ricorda l'ovvio fatto che la tua memoria è un flusso continuo di byte unidimensionali. Una matrice 2D è memorizzata in memoria per righe:

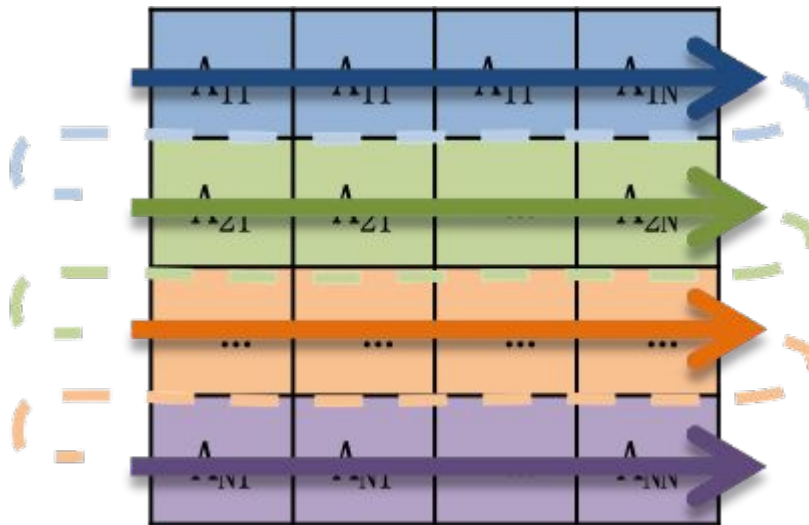


Questa convenzione è la convenzione C/C++, etichettata come **ordine row-major**. Si noti che la convenzione Fortran è opposta, con le colonne contigue in memoria (*column-major*).

Nota: come una matrice viene memorizzata nella memoria

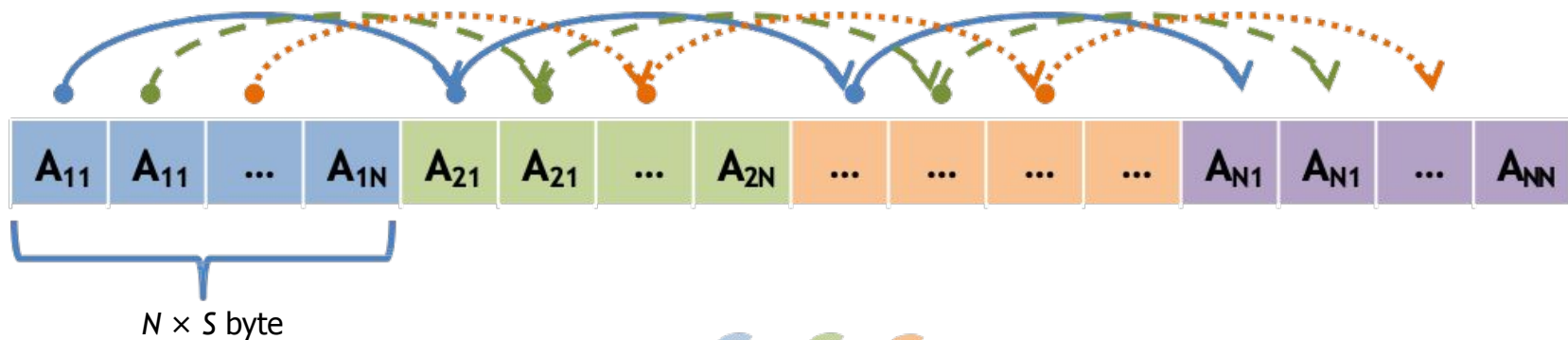


Quindi, attraversare la matrice nello stesso ordine di riga principale corrisponde ad attraversare la memoria in ordine contiguo.

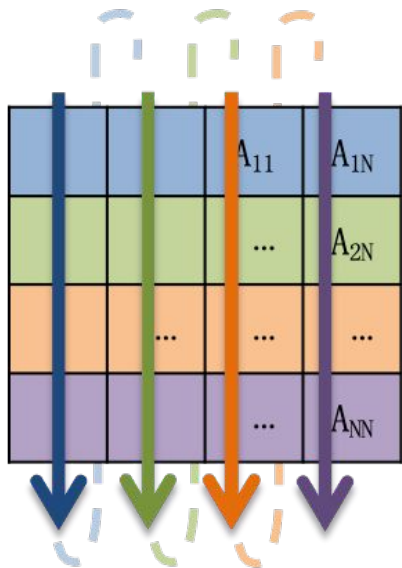


Leggere la matrice in questo ordine è molto meglio perché sono sequenziali. Al contrario, leggerli dalle colonne consuma molto di più.

Nota: come una matrice viene memorizzata nella memoria



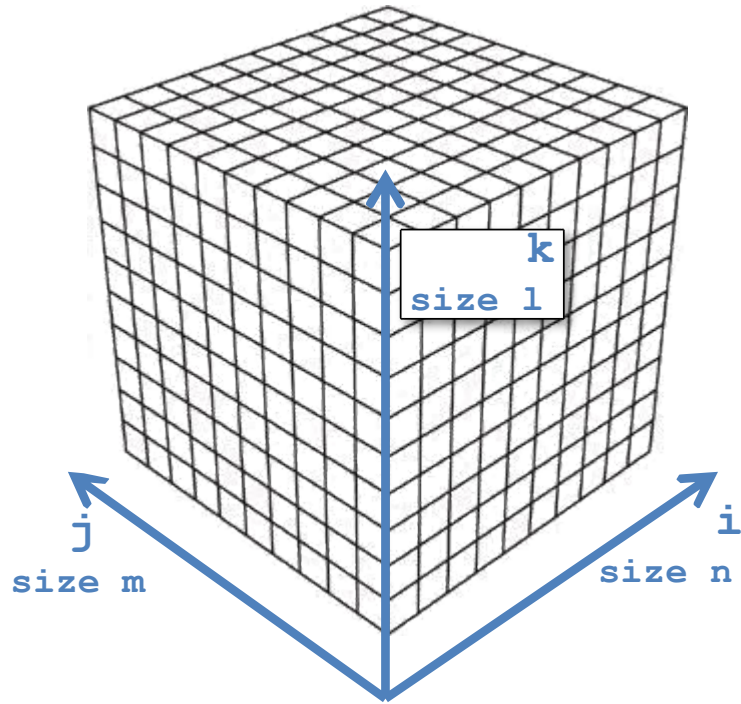
Mentre attraversare la matrice nell'ordine opposto della colonna maggiore equivale a saltare nella memoria di N posizioni, ovvero $N \times S$ byte, dove S è la dimensione di ciascun elemento.



Nota: come una matrice viene memorizzata nella memoria

viene memorizzata nella memoria una matrice 3D, indicizzata come $[i][j][k]$?

```
double M[n][m][l]
```



L'indice più interno è quello che varia più velocemente e traccia la contiguità della memoria.

Quindi $M[i][j][k]$ è contiguo a $M[i][j][k+1]$.

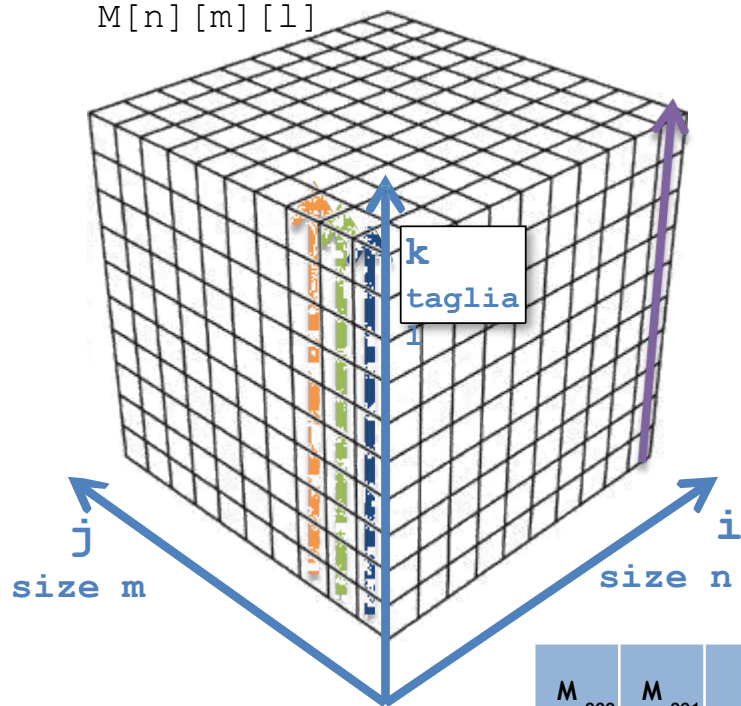
Nota: come una matrice viene memorizzata nella memoria

viene memorizzata nella memoria una matrice 3D, indicizzata

come $[i][j][k]$?

double

$M[n][m][l]$



L'indice più interno è quello che varia più velocemente e traccia la contiguità della memoria.

Quindi $M[i][j][k]$ è contiguo a $M[i][j][k+1]$.

Nella scelta che abbiamo fatto qui, la matrice 3D viene quindi memorizzata lungo la direzione z , che è il nostro indice k .

Poi per fette lungo la direzione y , che è il nostro indice j . E infine lungo la direzione x , che è il nostro indice i , il più lento nel cambiare.

Per fare riferimento direttamente alla voce $[i][j][k]$ in M si dovrebbe usare

$$M + i * (m * l) + j * l + k$$





Accesso a grandi passi

L'implementazione molto semplice e diretta è:

```
for(int row = 0; row < N;  
row++)  
    for(col = 0; col < N;  
col++)
```

**Trasposta della
matrice**

$$\mathbf{A} [\text{col} * \mathbf{N} + \text{row}] = \mathbf{B} [\text{row} * \mathbf{N} + \text{col}] ;$$

Nota: l'accesso a grandi passi ad A o B è inevitabile

Tuttavia: è meglio averlo in *lettura o in scrittura?*





Accesso a grandi passi

L'implementazione molto semplice e diretta è:

```
for(int row = 0; row < N;  
    row++)
```

Strided write

- lettura contigua
- scrittura con stride

```
    for(col = 0; col < N;  
        col++)
```

Contiguous write

- scrittura contigua
- lettura con stride

A [col*N + row] = **B** [row*N + col];

oppure

A [row*N + col] = **B** [col*N + row];

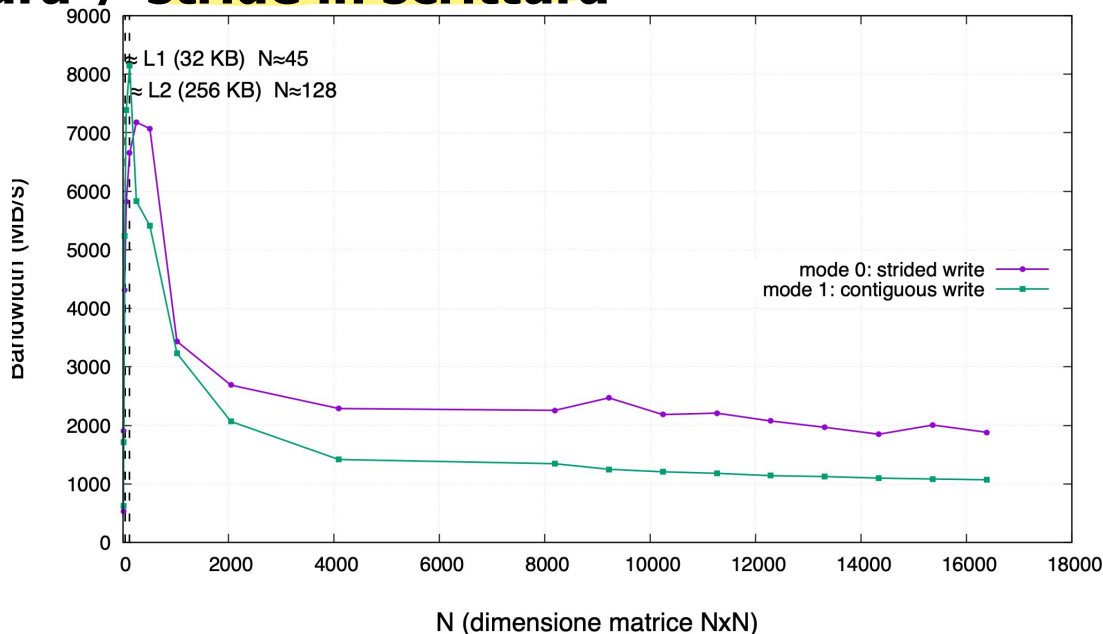




Cache e
memoria

Accesso a grandi passi

In entrambi i casi c'è **uno stride**. Ma:
stride in lettura $\neq$ stride in scrittura





Gestire le scritture

Quando si modifica il contenuto di una variabile, tale modifica equivale a sovrascrivere una posizione di memoria.

Quando la variabile viene caricata nella memoria cache, ciò che viene caricata è una *copia* della variabile, che mantiene la sua posizione originale nella memoria principale.

Quindi, se modifichiamo il suo valore, dobbiamo modificarlo solo nella cache o anche nella memoria principale?

ci sono 2 possibili politiche:

1) **scrittura completa**

la memoria e la cache vengono mantenute coerenti, quindi il valore viene immediatamente riscritto nella memoria

2) **risrittura**

il nuovo valore viene memorizzato solo nella cache e viene propagato ai livelli inferiori quando la riga della cache viene sostituita.



Gestire le scritture

La scrittura complica molto le cose nella cache, a seconda della politica implementata dalla cache.

In caso di *write-miss* in una cache write-through, è possibile allocare un blocco nella cache (*write-allocate*) oppure non allocare e scrivere direttamente nella memoria principale (*non-write-allocate*). In quest'ultimo caso, non è necessario alcun recupero dei dati. La modalità non-write-allocate è utile per le operazioni di scrittura a raffica.

Se in una cache write-back si , è necessario prima riscrivere il blocco in memoria



Accesso a grandi passi

L'implementazione molto semplice e diretta è:

```
for(int row = 0; row < N;  
row++)  
    for(col = 0; col < N;  
col++)
```

**Trasposizione
della matrice**

$$\mathbf{A} [\text{col} * \mathbf{N} + \text{row}] = \mathbf{B} [\text{row} * \mathbf{N} + \text{col}];$$

Nota: l'accesso a grandi passi ad A o B è inevitabile

Tuttavia: è meglio averlo in *lettura* o in *scrittura*?



Accesso a grandi passi

Versione ingenua:

```
for(int riga = 0; riga < N;  
    riga++) for(col = 0; col < N;  
    col++)
```

**Trasposizione
della matrice**

$$\mathbf{A}[\text{col} \cdot \mathbf{N} + \text{riga}] = \mathbf{B}[\text{riga} \cdot \mathbf{N} + \text{col}]$$

Tuttavia: è meglio averlo in lettura o in scrittura?
A causa delle transazioni di scrittura-allocazione nella cache, le scritture con avanzamento sono più costose dei caricamenti con avanzamento.





Cache e
memoria

Esplorare i dati per migliorare la località

2 esempi:

1) blocco

2) curva z di Morton



Organizzazione dei dati per migliorare la località

Ordine della
linea di
scansione
row-major

(a) ~~0~~ ~~1~~ ~~2~~ ~~3~~ ~~4~~ ~~5~~ ~~6~~ ~~7~~
~~8~~ ~~9~~ ~~10~~ ~~11~~ ~~12~~ ~~13~~ ~~14~~ ~~15~~
~~16~~ ~~17~~ ~~18~~ ~~19~~ ~~20~~ ~~21~~ ~~22~~ ~~23~~
~~24~~ ~~25~~ ~~26~~ ~~27~~ ~~28~~ ~~29~~ ~~30~~ ~~31~~
~~32~~ ~~33~~ ~~34~~ ~~35~~ ~~36~~ ~~37~~ ~~38~~ ~~39~~
~~40~~ ~~41~~ ~~42~~ ~~43~~ ~~44~~ ~~45~~ ~~46~~ ~~47~~
~~48~~ ~~49~~ ~~50~~ ~~51~~ ~~52~~ ~~53~~ ~~54~~ ~~55~~
~~56~~ ~~57~~ ~~58~~ ~~59~~ ~~60~~ ~~61~~ ~~62~~ ~~63~~

Ordine in
blocco
+ scansione
sub-ordine

(c) ~~0~~ ~~1~~ ~~2~~ ~~3~~ ~~16~~ ~~17~~ ~~18~~ ~~19~~
~~4~~ ~~5~~ ~~6~~ ~~7~~ ~~20~~ ~~21~~ ~~22~~ ~~23~~
~~8~~ ~~9~~ ~~10~~ ~~11~~ ~~24~~ ~~25~~ ~~26~~ ~~27~~
~~12~~ ~~13~~ ~~14~~ ~~15~~ ~~28~~ ~~29~~ ~~30~~ ~~31~~
~~32~~ ~~33~~ ~~34~~ ~~35~~ ~~48~~ ~~49~~ ~~50~~ ~~51~~
~~36~~ ~~37~~ ~~38~~ ~~39~~ ~~52~~ ~~53~~ ~~54~~ ~~55~~
~~40~~ ~~41~~ ~~42~~ ~~43~~ ~~56~~ ~~57~~ ~~58~~ ~~59~~
~~44~~ ~~45~~ ~~46~~ ~~47~~ ~~60~~ ~~61~~ ~~62~~ ~~63~~

(b) 0 8 16 24 32 40 48 56
1 9 17 25 33 41 49 57
2 10 18 26 34 42 50 58
3 11 19 27 35 43 51 59
4 12 20 28 36 44 52 60
5 13 21 29 37 45 53 61
6 14 22 30 38 46 54 62
7 15 23 31 39 47 55 63

Ordine della linea
di scansione
col-major



Attraversamento delle matrici tramite blocchi

Una tecnica comune nella moltiplicazione matrice-matrice $A \times B = C$, o nell'inversione di matrice, consiste nell'elaborare le matrici per blocchi anziché attraversare intere righe/colonne.

Ovviamente, se la dimensione del blocco viene scelta saggiamente, ciò aumenta la possibilità che tutti e 3

i blocchi (da A, B e C) siano ospitati in L1

~~2 3 16 17 18 19~~
~~14 5 6 7 20 21 22 23~~
~~8 9 10 11 24 25 26 27~~
~~12 13 14 15 28 29 30 31~~
~~32 33 34 35 48 49 50 51~~
~~36 37 38 39 52 53 54 55~~
~~40 41 42 43 56 57 58 59~~
44 45 46 47 60 61 62 63

```
void mblock (int n, int bsize,
             double *A, double *B, double *C)
{
    for (int ii = 0; ii < bsize; ++ii)
        for (int jj = 0; jj < bsize; ++jj) {
            int jjn = jj*n;
            double cij = C[jjn + ii];           /* cij = C[i][j] */

            for( int kk = 0; kk < bsize; kk++ )
                cij += A[ii+kk*n] * B[kk+jjn]; /* cij += A[i][k]*B[k][j] */

            C[ii+jjn] = cij; }                  /* C[i][j] = cij */
}

void dgemm (int n, int bsize, double* A, double* B, double* C)
/* C_ij = SUM A_ik * B_kj */
{
    for ( int j = 0; j < n; j += bsize )
        for ( int i = 0; i < n; i += bsize )
            for ( int k = 0; k < n; k += bsize ) {
                double *AA = A + k*n + i;
                double *BB = B + j*n + k;
                double *CC = C + i*n + j;
                mblock(n, bsize, AA, BB, CC); }
}
```



Organizzazione dei dati per migliorare la località

Ordine della
linea di
scansione
row-major

(a) 0 1 2 3 4 5 6 7
8 9 10 11 12 13 14 15
16 17 18 19 20 21 22 23
24 25 26 27 28 29 30 31
32 33 34 35 36 37 38 39
40 41 42 43 44 45 46 47
48 49 50 51 52 53 54 55
56 57 58 59 60 61 62 63

Ordine in
blocco
+ scansione
sub-ordine

(c) 0 1 2 3 16 17 18 19
4 5 6 7 20 21 22 23
8 9 10 11 24 25 26 27
12 13 14 15 28 29 30 31
32 33 34 35 48 49 50 51
36 37 38 39 52 53 54 55
40 41 42 43 56 57 58 59
44 45 46 47 60 61 62 63

(b) 0 8 16 24 32 40 48 56
1 9 17 25 33 41 49 57
2 10 18 26 34 42 50 58
3 11 19 27 35 43 51 59
4 12 20 28 36 44 52 60
5 13 21 29 37 45 53 61
6 14 22 30 38 46 54 62
7 15 23 31 39 47 55 63

Ordine della linea
di scansione
col-major

Cosa succederebbe se fosse possibile generare lo stesso schema by-blocks "naturalmente", senza doverlo codificare nel codice, il che comporta una regolazione precisa di un parametro (il valore della dimensione del blocco) su una determinata piattaforma?

Le prestazioni potrebbero non essere ottimali come con una messa a punto precisa, ma le prestazioni medie su tutte le piattaforme potrebbero risultare molto buone.



Organizzazione dei dati per migliorare la località

Ordine della
linea di
scansione
per-riga

(a) 0 1 2 3 4 5 6 7
8 9 10 11 12 13 14 15
16 17 18 19 20 21 22 23
24 25 26 27 28 29 30 31
32 33 34 35 36 37 38 39
40 41 42 43 44 45 46 47
48 49 50 51 52 53 54 55
56 57 58 59 60 61 62 63

(b) 0 8 16 24 32 40 48 56
1 9 17 25 33 41 49 57
2 10 18 26 34 42 50 58
3 11 19 27 35 43 51 59
4 12 20 28 36 44 52 60
5 13 21 29 37 45 53 61
6 14 22 30 38 46 54 62
7 15 23 31 39 47 55 63

Ordine della linea
di scansione
per-colonna

Ordine in blocco
+ scansione
sub-ordine

(c) 0 1 2 3 16 17 18 19
4 5 6 7 20 21 22 23
8 9 10 11 24 25 26 27
12 13 14 15 28 29 30 31
32 33 34 35 48 49 50 51
36 37 38 39 52 53 54 55
40 41 42 43 56 57 58 59
44 45 46 47 60 61 62 63

(d) 0 1 4 5 16 17 20 21
2 3 6 7 18 19 22 23
8 9 12 13 24 25 28 29
10 11 14 15 26 27 30 31
32 33 36 37 48 49 52 53
34 35 38 39 50 51 54 55
40 41 44 45 56 57 60 61
42 43 46 47 58 59 62 63

Ordine Z o *bit
interlacciato* o
Ordine di
Morton



Cache e
memoria

Organizzare i dati per migliorare la località

2 esempi :

- 1) campi *caldi* e *freddi*
- 2) le curve che riempiono lo spazio (di nuovo, ma in modo diverso)



Organizzazione dei dati per migliorare la località

Quando la larghezza di banda della memoria è "limitata", come può essere il caso dei sistemi altamente paralleli + multicore con una gerarchia NUMA di stringhe, **l'ottimizzazione della località dei dati** può svolgere un ruolo importante.

La riorganizzazione dei dati nello "spazio" (qualunque sia il loro spazio n -dimensionale) in modo che il modello di accesso sia ottimale per un dato algoritmo è correlata a tale ottimizzazione della località.



Campi caldi e freddi

Riordinare i campi nelle strutture in modo che ciò che viene utilizzato insieme rimanga insieme

Nodo lista collegata

```
struct my_node {  
    double    key;  
    char      my_data[300];  
    my_node *next_node; }
```

p->key e p->next_node sono usati assieme, quindi è giusto che siano vicini anche nella struttura. Così facendo saranno salvate in maniera consecutiva anche nella cache

Attraversamento di liste collegate

```
void myfunc(my_node *p, double key, <...>)  
{  
    while( p != NULL) {  
        if( p->key == key ) {  
            do_something( <.> );  
            break;  
        }  
        p = p -> next_node;  
    }  
}
```



Campi caldi e freddi

Riordinare i campi nelle strutture in modo che ciò che viene utilizzato insieme rimanga insieme

Poiché stiamo cercando un nodo univoco nell'elenco, quello che ha la *chiave* che stiamo cercando, tutti eccetto un nodo vengono scartati nella nostra ricerca.

Quindi, il solito schema di esecuzione è

```
while(p != NULL) {  
    if(p->chiave ==  
        chiave) {} p = p ->  
        next_node; }
```

In altre parole, i campi `key` e `next_node` sono temporaneamente locali, nel senso che vengono utilizzati uno dopo l'altro.

Attraversamento di liste collegate

```
void myfunc(my_node *p, double key,  
<...>)  
{  
    while( p != NULL) {  
        if( p->key == key )  
        { do_something( <...>  
            ); break;  
        }  
        p = p -> next_node;  
    }  
}
```



Campi caldi e freddi

Riordinare i campi nelle strutture in modo che ciò che viene utilizzato insieme rimanga insieme

Tuttavia, ci sono 300 byte tra `key` e `next_node`, il che non è un modo ottimale di organizzare i dati perché sicuramente non sono spazialmente locali né in memoria né in cache, mentre sono temporalmente locali.

```
struct my_node
{
    double    key;
    char      my_data[300];
    my_node *next_node;
}
```



Campi caldi e freddi

Riordinare i campi nelle strutture in modo che ciò che viene utilizzato insieme rimanga insieme

```
struct my_node
```

```
{
```

```
    double    key;
```

```
    char      my_data[300];
```

```
    my_node *next_node;
```

```
}
```



```
struct my_node
```

```
{
```

```
    double    key;
```

```
    my_node *next_node;
```

```
    char      my_data[300];
```

```
}
```

Una prima mossa è semplicemente quella di scambiare la posizione di `my_data` e `next_node`.

Questo è un semplice esempio, per chiarire cosa significa ottimizzare il layout dei dati per la località della cache.



Campi caldi e freddi

Riordinare i campi nelle strutture in modo che ciò che viene utilizzato insieme rimanga insieme

Tuttavia, il vero problema è il gruppo di dati `my_data` .

Una mossa molto significativa è quella di separare la chiave *dei metadati* e `next_node` dai *dati* `my_data` , a cui si accede solo dopo che la ricerca nell'elenco collegato ha esito positivo. Durante l'attraversamento della lista concatenata, i campi `key` e `next_node` sono i campi *attivi* , mentre `my_data` è un campo *freddo*.

In generale, una buona strategia è quella di mantenere i campi caldi (ovvero i dati temporalmente vicini) il più possibile vicini spazialmente.

```
struct my_node
{
    double    key;
    my_node  *next_node;
    char      my_data[300];
}
```




Campi caldi e freddi

Dividere i campi in modo da mantenere consecutivi i campi utilizzati in sequenza

```
struct my_node
{
    double    key;
    char
my_data[300];
    my_node *next_node;
}
```

Anziché tenere my_data[300] (quindi memorizzarlo tutto nella struct e, quindi, in cache. Manteniamo solamente un puntatore così da non dover salvare tutti i dati in memoria.



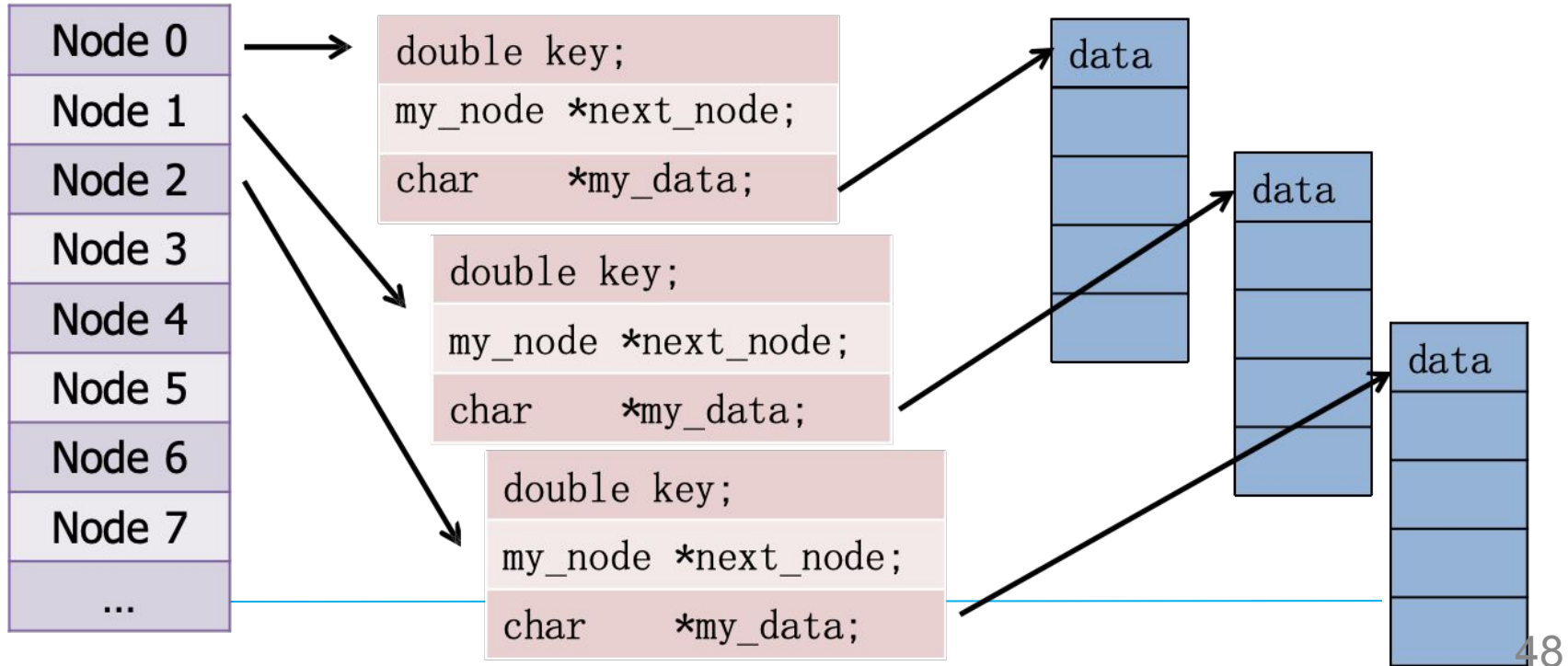
```
struct my_node
{
    double    key;
    my_node *next_node;
    void      *my_data;
}
```

```
struct my_data
{
    char data[300];
}
```



Campi caldi e freddi

Questi sono chiamati campi *caldi* e *freddi*





Organizzazione dei dati per migliorare la località

Nello “spazio” in cui risiedono i dati – ad esempio il nostro consueto spazio 3D – potrebbe esserci una metrica correlata alla coerenza spaziale.

In genere è più probabile accedere in un breve lasso di tempo a punti che sono anche spazialmente “vicini”.

Quindi, due strategie ovvie per sfruttare questo sono

- Ridurre al minimo la distorsione della distanza
- Preservare la località

ovvero **mantenendo vicini nel mondo della memoria 1D i punti che sono vicini in n - dimensioni aumenta la probabilità di utilizzare posizioni di memoria vicine mentre sono ancora nella cache .**



Messaggio principale sulla cache

Il modo in cui si posizionano i dati nella memoria e il modo in cui vi si accede sono di fondamentale importanza.

Ciò ha un impatto diretto sul *modello di dati* che scegli e implementi e sul modo in cui progetti il tuo flusso di lavoro.

Approfondiremo i dettagli nelle prossime lezioni...
